

Large Language Models (MGL-GIA) Course 2025/26

Group Project: Aspect Based Sentiment Analysis with LLMs

(March 2026)

1. Introduction

E-commerce, as a new shopping and marketing channel, has led to an upsurge of review sites for a variety of services and products. In this context, **Aspect Based Sentiment Analysis** (ABSA)—i.e. mining and summarizing opinions from text about specific entities and their aspects—can help consumers decide what to purchase and businesses to better monitor their reputation and understand the needs of the market.

Given a target of interest (i.e. a product or a business such as a restaurant) an ABSA system can extract from the text how each users values its different aspects, allowing detailed automatic scoring (e.g., in a 1-5 stars scale).

2. Dataset for the MG-GIA lab project

The target dataset was introduced in a SemEval task in 2014 (SE-ABSA14) providing benchmark datasets of English reviews and a common evaluation framework (Pontiki et al., 2014); the datasets were annotated with polarity for laptop and restaurant reviews, as well as coarser aspect categories (e.g., FOOD) and their polarity.

The task was repeated in SemEval 2015 (SE-ABSA15) aiming to facilitate more in-depth research by providing a new ABSA framework in which all the identified constituents of the expressed opinions (aspects, opinion target expressions and sentiment polarities) meet a set of guidelines/specifications and are linked to each other within tuples.

We will be using a simplified version of SemEval 2016 (SE-ABSA16), where the review is treated as a whole (not in separate sentences as in previous editions), and the goal is to extract the **sentiment polarity for each aspect mentioned in the review**. We will focus on restaurant reviews in English and Spanish so the task is multilingual.

For instance, for the following restaurant review (which is natural language, including typos, grammatical and punctuation mistakes):

The food here is rather good, but only if you like to wait for it. I like the somosas, chai, and the chole, but the dhosas and dhal were kinda dissapointing. The service varys from day to day- sometimes they're very nice, and sometimes not. The kitchen however, is almost always slow. Also, specify if you like your food spicy- its rather bland if you don't. The ambience is pretty and nice for conversation, so a casual lunch here would probably be best.

The system would be expected to extract the following opinions:

- **aspect:** food_quality
polarity: conflict
("I like the somosas, chai, and the chole, but the dhosas and dhal were kinda dissapointing")

- **aspect:** service
polarity: negative
(*"The service varies from day to day- sometimes they're very nice, and sometimes not."*)
- **aspect:** ambience
polarity: positive
(*"The ambience is pretty and nice for conversation"*)
- **aspect:** restaurant_general
polarity: positive
(*"a casual lunch here would probably be best"*)

The list of aspects and polarity values are predefined and closed. See the list below:

Possible aspects:

- *restaurant_general*: opinions about general aspects of the restaurant.
- *restaurant_prices*: opinions about price of the restaurant.
- *food_quality*: opinions about food quality.
- *food_prices*: opinions about food prices.
- *food_style_options*: opinions about food style and options: presentation, originality, variety, portion size, etc.
- *drinks_quality*: opinions about drink quality.
- *drinks_prices*: opinions about drink prices.
- *drinks_style_options*: opinions about drink style and options: presentation, originality, variety, portion size, etc.
- *ambience*: opinions about the restaurant atmosphere.
- *service*: opinions about the service (kindness, promptness, attitude, professionalism, etc).
- *location*: opinions about the location of the restaurant (position, surroundings, view, etc).

Possible polarity values for each aspect:

- *positive*: the aspect is evaluated positively (e.g., "yummy food").
- *negative*: the aspect is evaluated negatively (e.g., "oil-soaked bland pizza").
- *neutral*: the aspect is evaluated with no strong feeling (e.g., "the food is ok, nothing special").
- *conflict*: the aspect has both positive and negative parts (e.g., "pizza was great, but meat was excessively cooked").

The dataset is provided in a json structure, containing a list of reviews. Each review is a JSON dictionary containing a review identifier (*rid*), the language of the review (*language*), the text of the review (*text*), and a dictionary with the mentioned aspects and their polarity (*opinions*).

For example:

```
{
  "rid": "1131595",
  "language": "en",
  "text": "This place is incredibly tiny.They refuse to seat parties of 3 or more on weekends. The hostess is rude to the point of being offensive. The food was bland oily. I just don't understand all the hype...",
  "opinions": {
```

```
"service": "negative",  
"food_quality": "negative",  
"restaurant_general": "negative"  
}  
}
```

The dataset is divided in three partitions:

- `train.json`: Reviews used to instruct, tune, or train the model. The model is shown the text and the expected opinions to be extracted.
- `devel.json`: Reviews used to evaluate the model performance during development. The model is given the text and asked to extract the opinions. The result is compared with the ground truth data to assess model performance. Decisions are taken (change of hyperparameters, prompts, etc) to improve the score.
- `test.json`: Reviews used once the best model has been selected based on its performance on development dataset. The model is given the text and asked to extract the opinions. The result is compared with the ground truth data to assess model performance and check whether the model is able to generalize to unseen data. *This partition will be disclosed towards the end of the semester*

3. Tasks for MML lab project

The goal of the project is to use relatively small and open source LLM (e.g., llama3.2 3B) via different strategies to extract aspect sentiments from restaurant reviews, and evaluate the performance of each strategy, as well as their advantages and drawbacks.

We propose to run four different tasks:

- **TASK 1. Zero shot prompting:** try to solve the task by just prompting an SLM with appropriate instructions.
- **TASK 2. Few-shot prompting:** complement task 1 by adding a few examples (a.k.a. *demonstrations*) in the prompts.
- **TASK 3. Data augmentation:** the training dataset is relatively small; can you use powerful LLMs to generate additional high-quality synthetic examples?
- **TASK 4. Fine-tuning:** adapt (specialize) the SLM to the task by fine-tuning it using the training set and the data augmentations.

We have prepared Python code and scripts for you to guide you through all the previous tasks, including the evaluation scripts. Some of the demo scripts will run a parallel task of Named Entity Recognition.

As computing resources, we will provide access to the Boada cluster, which has several GPUs that will support all the experiments you need to run for this project. We have tested them and response times are within reasonable bounds. The environment at Boada has installed all necessary libraries and there are no missing dependencies with the code we provide. We will make available a user guide for Boada as well as part of the shared distribution. If you have access to alternative computing resources and you want to run your project outside Boada, you can do it. This is not a mandatory path.

The following is the list of steps that you can run to get familiar with the setup and to run the projects tasks step by step. All the following is intended to help you to complete the necessary steps, but if you want to run the project from scratch you are also welcome to do so.

TASK 1: Zero shot prompting

- Check the program `zeroshot.py` which performs a named entity recognition task on news texts using a zero-shot approach. Execute it and observe the produced result files.
`$ sbatch zeroshot.sh`
- Adapt the `zeroshot.py` program to get the prompts from a json file given as argument.
- Create a file with prompts describing the target ABSA task, in a similar way than the NER task is described (i.e., in detail and precisely).
- Run the `zeroshot.py` with the ABSA `devel.json` dataset. Evaluate results with `stats.py` script.
`$ sbatch zeroshot.sh myprompts.json`
- Analyze the results and adapt your prompts to lead the model towards the desired behaviour. Evaluate results with `stats.py` script.
- Try different parameter values for the call to `model.generate`. Usual tunable parameters are *temperature* and *top_p*. Evaluate results with `stats.py` script.
- Iterate over different combinations of parameters and prompts. Evaluate each combination. Keep the best.
`$ sbatch zeroshot.sh myprompts.json <temp> <top_p>`

(note: you can consider averaging different runs with the same parameterization; same for task 2)

TASK 2: Few shot prompting

- Copy the program `zeroshot.py` to a new file `fewshot.py`. Modify the program so it accepts a new argument (apart from the prompt file and others you may have) with the desired number of shots *NS*.
- Adapt the program so that the prompt each created for each example includes *NS* examples of user-assistant interactions just after the «system» message and before the «user» message with the question to be answered.

To do so, modify the function `common.prepare_messages` to receive as a new parameter a list of the train example to be added to the prompt. The *NS* examples must be selected from the train dataset partition (`train.json`). Extract the desired sample of examples, and pass it to `common.prepare_messages` in a new parameter, so the function can incorporate the few-shot examples to the prompt.

- Run `fewshot.py` with the ABSA `devel.json` dataset. Evaluate results with `stats.py` script.
`$ sbatch fewshot.sh myprompts.json <num_shots> <temp> <top_p>`
- Try different strategies to select the *NS* examples:
 - Random (e.g. first *NS*, random sample –with a seed–, ...)
 - Cheery picked (select examples that show variety and different possible responses, covering as much aspect/polarity combinations as possible)

- Try with different number of examples *NS* (e.g., 5, 10, 20, 30,...). See where the results plateau.
- Try different parameter values for the call to `model.generate`. Usual tunable parameters are *temperature* and *top_p*. Evaluate results with `stats.py` script.
- Iterate over different combinations of parameters, prompts, and *NS*. Evaluate each combination. Keep the best.
- Repeat experiments loading the model quantized in 4 bits. Evaluate results.
- Elaborate comparative tables with M.avg F1 for each tried parameterization

TASK 3: Data augmentation

- Write a script to create 1,000 random combinations of aspect+polarity sets in the same json format than the «gold» entry in the dataset examples.
- Use the `fewshot.py` program with a new set of prompts: Now you ask the model to generate possible reviews that match given «gold» json.

That is, the «system» prompt describes the task and the «user» prompt summarizes it and asks the model to apply it to given opinions json. After the «system» prompt, several user-assistant pairs provide examples (taken from train dataset) of a json (user) and a possible sentence for it (assistant). Finally a user prompt with the target json will prompt the model to continue, generating a plausible review. Repeating this 1,000 times, you'll get 1000 new examples of reviews and their json.

- Manually check the obtained examples (or at least sample a few dozens) in order to assess their coherence and have an estimate of their correctness. If the sentences do not reflect the opinions in the json, try again improving the prompt, or changing some hyperparameter (temperature, *top_p*, etc.)
- When the new examples are generated, store them in a file `synt.json` with the same format than `train.json`.

TASK 4: Fine-tuning

- Fine-tune the base model with the original `train.json` dataset, using `devel.json` for validation. Evaluate results with `stats.py` script.

```
$ sbatch finetune-train.sh myprompts.json train.json
```
- Fine-tune the base model with the augmented dataset (`train.json` plus synthetic data), using `devel.json` for validation. Evaluate results with `stats.py` script.

```
$ sbatch finetune-train.sh myprompts.json train+synth.json
```
- Try different values for the training parameters. Usual tunable parameters are batch size (*per_device_train_batch*, *per_device_eval_batch*), *gradient_accumulation_step*, float encoding (*fp16/bf16*), *learning_rate*, *epochs*, etc. Evaluate results with `stats.py` script.
- Execute the obtained model on `devel.json` dataset. Evaluate results with `stats.py` script.

```
$ sbatch finetune-inference.sh myprompts.json
```
- Repeat experiments loading the model quantized in 4 bits. Evaluate results with `stats.py` script.

- In inference, try different parameter values for the call to `model.generate`. Usual tunable parameters are *temperature* and *top_p*. Evaluate results with `stats.py` script.

```
$ sbatch finetune-inference.sh myprompts.json <temp> <top_p>
```

4. Project rules, dates, and evaluation

- The teams will be composed by 2 persons (exceptions will be considered) and they need to be notified to the course instructor **no later than March 19** (email is fine).
- From March 12 to the end of the quadrimester we will be using lab hours (Thursdays 14:00-16:00) to work practically on the project.
- The resources provided (Boada, code and scripts, LLMs, etc.) are optional. If you have alternative means to run the project that are preferable to you, please go ahead but notify this fact to the course instructor.
- The coverage of tasks 1-4 is mandatory, but the students can voluntarily add more tasks and variations if so desired. Any extension should be notified to the course instructor. Keep in mind that the total workload of the project has been quantified to be 25h of autonomous learning per student.
- Assessment of the project:
 - You need to deliver a group report explaining the results and lessons learned from the 4 mandatory tasks on the ABSA corpus. Maximum of 10 pages. Do not explain the task, motivation, or any of the things covered already by this document. Explain only what you have done, and present the results of your experiments with an analysis and conclusions extracted from them.
 - You will also have an in-person (20-minute) discussion with the course instructor about the project. All members of the team have to be present for the in-person session. We'll run this in front of a terminal/computer so we can look at the code, evaluation results, logs, etc.
 - In due time the course instructor will provide the means for each team to reserve some time slot to discuss the project results, ideally during the last two lab sessions of the course.
 - Recall that the project grade counts 40% towards the course final grade. Grades will be the same for all the members of the team. They'll be notified before the final exam date.
- Additional evaluation and leaderboard:
 - A held-out test set will be released before the end of the course so students can evaluate their best model on these fresh data.
 - A leaderboard will be published at the end of the course with the results of all teams.
 - We expect a fair participation to this "competition". Do not leak test data into training or do not make manual adjustments. This is just for fun. **Grades will not depend on the team ranking in the leaderboard.**